Day → 4 → Continue....

Autoencoders

1. What is an Autoencoder?

An **Autoencoder** is a **neural network designed to learn compressed representations of input data and then reconstruct the original data from that representation**.

It belongs to **unsupervised learning** because it does not require labeled data.
Instead, the model learns by trying to reproduce the input.

Objective

The goal is to make the reconstructed output as close as possible to the original input. '

x \approx \hat{x} → $x \approx \bar{x}$

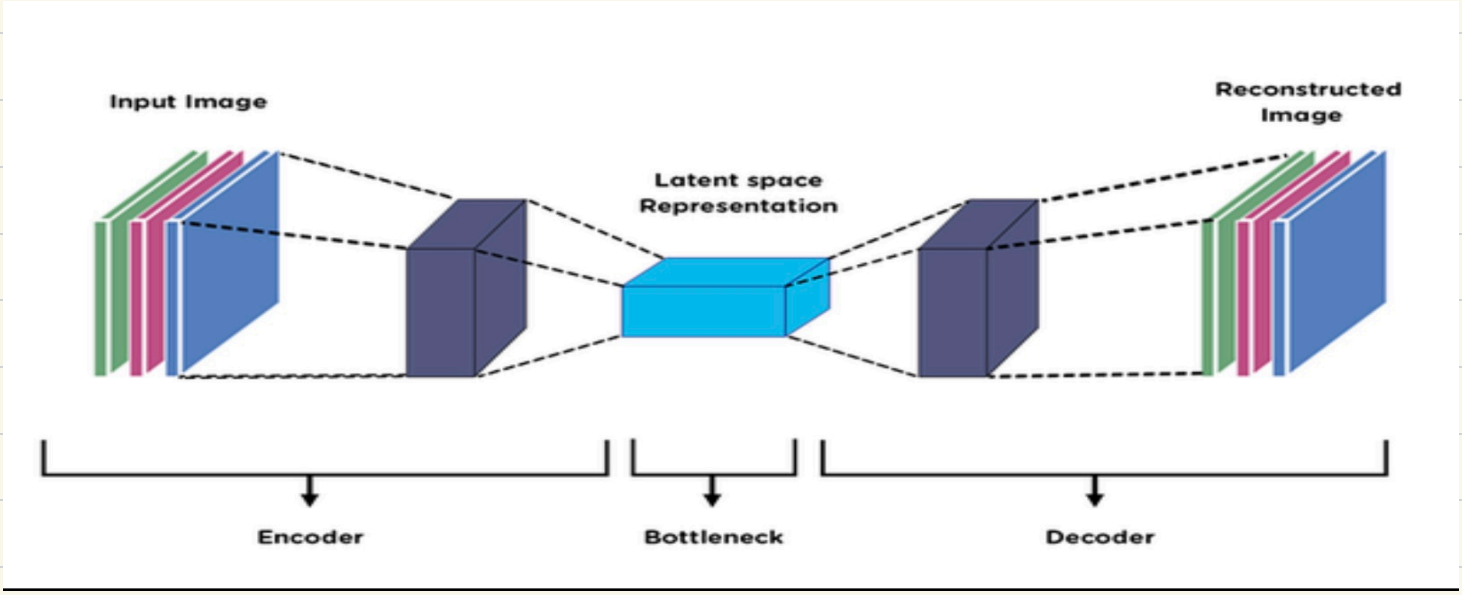
Where:

- **x** = original input
- \hat{x} = reconstructed output

($\bar{x}$)

The network learns **important patterns and features** in the data during this process.

2. Autoencoder Architecture



An autoencoder consists of **three main components**.

2.1 Encoder

The **encoder compresses the input into a lower-dimensional representation**.

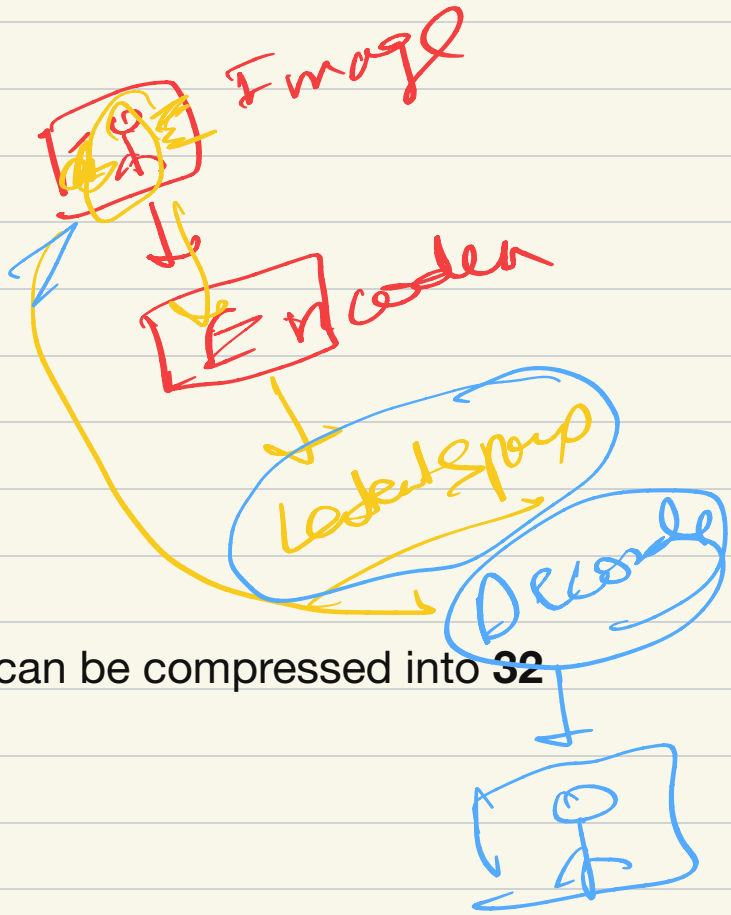
It extracts important features from the data and removes unnecessary information.

Mathematical representation:

$z = f(x)$

Where:

- **x** = input data
- **f** = encoder function
- **z** = compressed representation



Example:

A grayscale image with **784 pixels** can be compressed into **32 numbers**.

2.2 Latent Space (Bottleneck)

The **latent space** is the compressed representation created by the encoder.

Characteristics:

- Lower dimension than the input
- Stores only the most important features
- Forces the model to learn meaningful structure in the data

Example:

For a **face image**, latent space may capture:

- face shape
- eye position
- lighting conditions
- facial orientation

This layer is called the **bottleneck** because it restricts the information flow.

2.3 Decoder

The **decoder reconstructs the original input from the latent representation**.

Mathematical representation:

\hat{x} = g(z) $\bar{x} = g(z)$

Where:

- **g** = decoder function
- **z** = latent vector
- \hat{x} = reconstructed output

$(\bar{z})$

The decoder attempts to rebuild the input as accurately as possible.

Training Objective

Autoencoders are trained to **minimize reconstruction error**.

Loss Function:

Loss = ||x - \hat{x}|| → $\|x - \bar{x}\|$

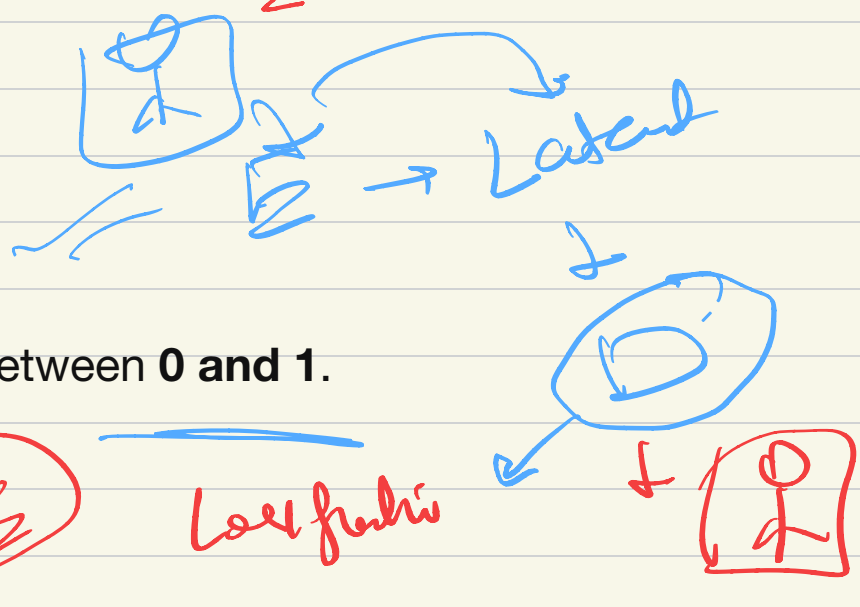
Common loss functions:

Mean Squared Error (MSE)

MSE = \frac{1}{n}\sum(x-\hat{x})^2 → $\frac{1}{2}\sum(x-\bar{x})^2$

Used for:

- image reconstruction
- continuous data



Binary Cross Entropy

Used when input values are between **0 and 1**.

Real-World Applications

1️⃣ Anomaly Detection

Used in:
- credit card fraud detection
- industrial fault detection

Idea:

Train on **normal data**.

If reconstruction error is high → anomaly.

2️⃣ Image Compression

Autoencoders can compress images much smaller than JPEG.

Used in:
- neural image codecs
- streaming

3️⃣ Feature Learning

Autoencoders learn **latent embeddings** used for:
- clustering
- classification

4️⃣ Medical Imaging

Detect abnormalities in:
- MRI scans
- X-rays

Latest Autoencoder Use Cases (2025–2026)

1️⃣ Video Compression for Streaming

Companies like **Netflix** use **deep autoencoder architectures** for neural video compression.

Benefits:

- smaller file sizes
- better streaming quality
- reduced bandwidth

2️⃣ Autonomous Driving Perception

Companies like **Tesla** use representation learning models similar to autoencoders for:

- scene understanding
- sensor compression
- anomaly detection

3️⃣ Medical Imaging AI

Hospitals use autoencoders to detect anomalies in MRI.

Example:

Brain tumor detection

Train on **normal brain scans**.

If reconstruction fails → possible tumor.

Variational Autoencoder (VAE)

1. What is a Variational Autoencoder?

A **Variational Autoencoder (VAE)** is a **generative deep learning model that can learn patterns from data and generate new similar data**.

It is an improved version of an **Autoencoder**.

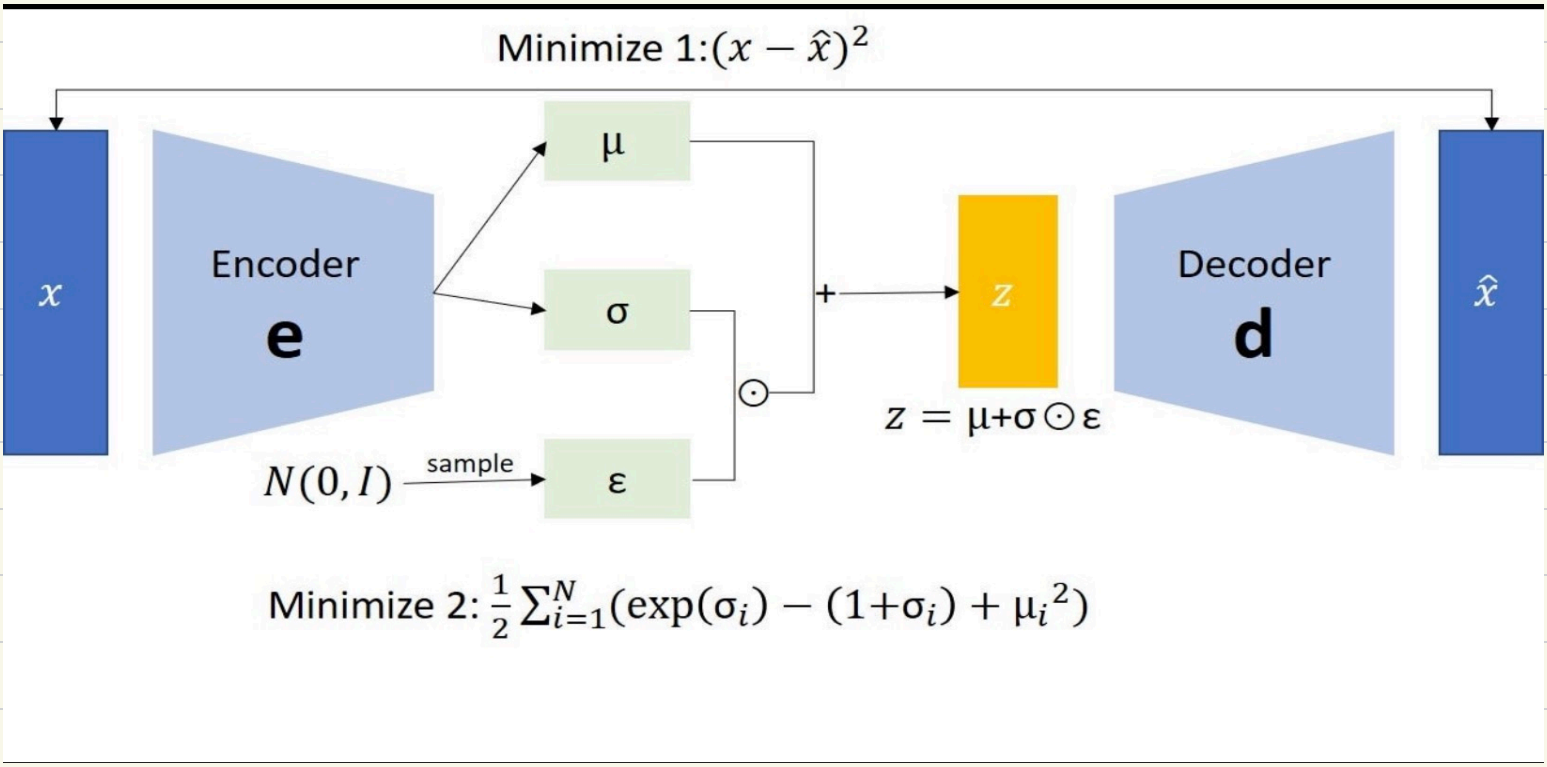
The key difference is:

- **Autoencoder** learns a compressed representation of data.
- **VAE** learns a **probability distribution of the data**, which allows it to **generate new samples**.

Example:

If we train a VAE on **handwritten digits**, it can generate **new digits that never existed in the dataset**.

VAE Architecture



A Variational Autoencoder has **three main parts**.

1. Encoder

The encoder reads the input data and converts it into **two values**:

- **Mean (μ)**
- **Variance (σ^2)**

These values describe a **probability distribution of the latent space**.

Instead of storing one compressed vector, the model learns **a distribution of possible representations**.

---

2. Latent Space (Sampling)

In VAE we **sample a latent vector** from the distribution.

Formula:

z = \mu + \sigma \times \epsilon $z = \mu + \sigma + \epsilon$

Where:

- **μ** = mean
- **σ** = standard deviation
- **ε** = random noise

This step allows the model to generate **different variations of data**.

3. Decoder

The decoder takes the sampled latent vector and **reconstructs the data**.

Output:

$\hat{x} = \text{Decoder}(z)$

The output should look **similar to the original input**.

---

3. Training Process

Training a VAE works like this:

1. Input data goes to the **encoder**
2. Encoder produces **mean and variance**
3. A latent vector is **sampled**
4. The **decoder reconstructs the input**
5. The model calculates the error
6. The network updates its weights using **backpropagation**

The model learns both:

- how to **reconstruct data**
- how to **organize the latent space distribution**

4. Loss Function

VAE uses **two losses**.

1. Reconstruction Loss

Measures how close the reconstructed output is to the input.

Loss_{recon} = ||x - \hat{x}|| $\Rightarrow \|x - \hat{x}\|$

---

2. KL Divergence

Ensures the latent space follows a **normal distribution**.

This makes it possible to generate new samples.

Total Loss

Loss = Reconstruction\ Loss + KL\ Divergence

---

5. Simple Example

Example: **Generating handwritten digits**

Training data: digits 0–9.

Process:

Digit image → Encoder → Mean & Variance → Sample latent vector → Decoder → Reconstructed digit.

After training, we can generate **new digits by sampling random latent vectors**.

6. Real Example in Modern AI

Models like **Stable Diffusion** developed by **Stability AI** use **VAE to compress images into latent space** before running diffusion models.

Benefits:

- Faster training
- Less memory usage
- Efficient image generation

---

7. One-Line Intuition

Autoencoder:

Compress → Reconstruct

VAE:

Learn distribution → Sample → Generate new data

Diffusion Models

1. What is a Diffusion Model?

A **Diffusion Model** is a **generative AI model that learns to create data by gradually removing noise from random noise**.

The idea comes from **diffusion in physics**, where particles spread over time. In AI, we do the opposite:

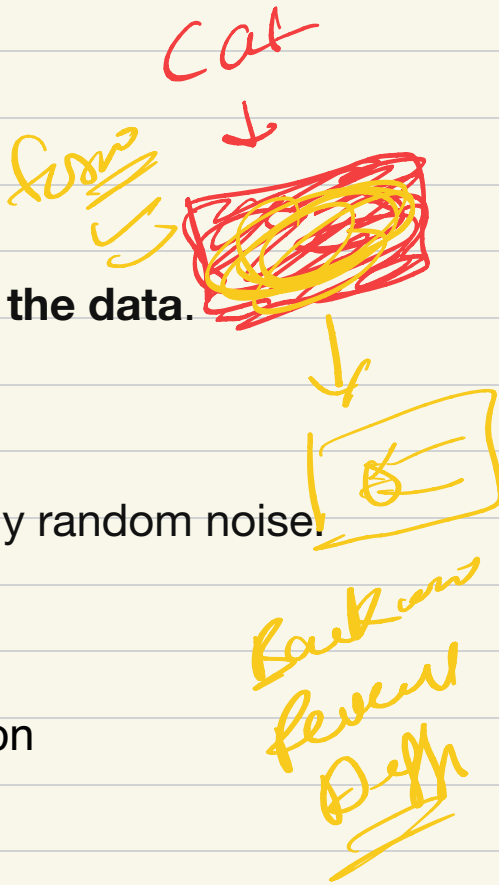
1. **Add noise to data step-by-step**
2. Train a model to **remove the noise step-by-step**

After learning this process, the model can start from **pure noise** and generate **new realistic images, audio, or videos**.

Example:

Random noise → gradually cleaned → realistic image.

Diffusion models work in **two main processes**.



1. Forward Diffusion (Adding Noise)

In this step, we **gradually add Gaussian noise to the data**.

Example with an image:

Image → slightly noisy → more noisy → completely random noise.

Mathematically:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon$$

Where:

- x_t = noisy image at step t
- x_{t-1} = previous step image
- ϵ = random noise

After many steps, the image becomes **pure noise**.

2. Reverse Diffusion (Removing Noise)

Now the model learns the **reverse process**.

It takes noisy data and predicts **how to remove the noise step-by-step**.

Process:

Noise → slightly clearer → clearer → final image.

The model learns this using a neural network (often a **U-Net architecture**).

---

3. Diffusion Model Architecture

The core component of diffusion models is a **denoising neural network**.

The network receives:

- Noisy image
- Time step t

And predicts:

- The noise present in the image.

Common architecture:

- **U-Net convolutional network**

Steps:

1. Input noisy image
2. Predict noise
3. Remove predicted noise
4. Repeat until a clean image appears

Simple Example

Suppose we train a diffusion model on **cat images**.

Generation process:

Random noise
↓
Remove noise step by step
↓
Shape appears
↓
Fur texture appears
↓
Final image: **cat**

The model creates a **new cat image that never existed in the dataset**.

---

7. Real Applications

Diffusion models are used in modern generative AI systems such as:

- **Stable Diffusion**
- **DALL·E**
- **Midjourney**

These models generate images from **text prompts**.

Example prompt:

"astronaut riding a horse on Mars"

The model starts with noise and gradually generates the image.

---

8. Why Diffusion Models Became Popular

Diffusion models became the **most powerful generative models for images** because they:

- produce **very high quality images**
- are **more stable than GANs**
- allow **controlled generation with text prompts**

This is why many modern systems like **Stable Diffusion** from **Stability AI** use diffusion models.